CLOUD & BLOCKCHAIN ARCHITECTURE SPECIFICATION

Multipurpose Supply Chain Management Platform

End-to-End Architectural Blueprint, Implementation Manual, AWS Cloud Services, and Cryptographic Blockchain Smart Contracts

---

Author / Lead Architect: Hardik Jha
Cloud Infrastructure: Amazon Web Services (DynamoDB, SQS, SNS, S3, IoT Core, IAM)
Blockchain Engine: SHA-256 Merkle Chain, Proof-of-Authority (PoA) / Amazon Managed Blockchain
Smart Contracts: Cold-Chain Compliance & Escrow Release Contract (SC-COLD-CHAIN-SLA)
Infrastructure as Code: HashiCorp Terraform (v1.5+)
Application Stack: Node.js v20 (Express), React 18, Vite, Tailwind CSS, Lucide Icons, Recharts
Repository: github.com/Deoid/Multipurpose_Supply_Chain_Management
Live URL: deoid.github.io/Multipurpose_Supply_Chain_Management/
Document Classification: Production Engineering Whitepaper (Technical Reference)

Architectural Directive: *This document provides an exhaustive, developer-grade breakdown of the Multipurpose Supply Chain Management system. It details why specific architectural decisions were chosen, how asynchronous event buses mitigate order spikes, how time-series IoT sensors enforce cold-chain compliance, how the cryptographic Blockchain ledger guarantees immutable chain-of-custody, and the complete security perimeter isolation model.*

1. Executive Problem Statement & Multipurpose Scope

1.1 The Operational Crisis in Modern Global Supply Chains

Modern enterprise supply chains face unprecedented challenges. Traditional supply chain management (SCM) software is overwhelmingly monolithic, synchronous, and siloed. When thousands of purchase orders arrive simultaneously during high-demand periods, relational database connections lock up, transaction timeouts occur, and orders drop without warning. Furthermore, for perishable items (such as pharmaceutical vaccines, biologics, and premium foods), lack of real-time environmental monitoring results in billions of dollars in thermal spoilage before products reach the receiving dock.

1.2 The 'Multipurpose' Architectural Requirement

A true 'multipurpose' SCM platform cannot be constrained to a single cargo type. Different industrial verticals have drastically different physical and data requirements. Our architecture unifies four major supply chain categories under one cloud platform:

Vertical	Physical Requirements	Cloud & Blockchain Enforcements
Pharmaceutical Cold-Chain	Strict 2°C to 8°C thermal buffer; ultra-cold -20°C to -80°C cryogenic storage for mRNA biologics.	AWS IoT rules evaluate continuous temperature streams. Deviations (>8°C or <2°C) trigger Amazon SNS alerts and freeze smart contract escrow on the blockchain.
High-Tech Semiconductors	Electrostatic Discharge (ESD) cleanrooms, low humidity, severe shock and vibration safeguards.	High-frequency accelerometer tracking. Shock events >4.5G trigger transit inspection warnings on the immutable ledger.
Industrial Machinery	Heavy servo motors, complex parts catalogues, multi-tier purchase orders, and long lead times.	Supplier SLA reliability ratings, automated purchase order creation, and immutable S3 document archiving.
Biotechnology Assays	Light-sensitive reagents, diagnostic PCR kits, batch-lot tracking with strict shelf-life expiry.	Dynamic safety stock calculations, low-stock reorder alerts, and cryptographically verified custody transfers.

2. Blockchain Ledger & Smart Contract Implementation

2.1 The Multi-Party Trust Problem in Global Logistics

In global supply chains, goods pass through manufacturers, freight carriers, customs checkpoints, and regional distribution hubs. Traditional databases are maintained by a single entity who has write access and can alter records after transit disputes occur. To establish absolute multi-party trust, our platform integrates an **Immutable Cryptographic Blockchain Ledger** powered by SHA-256 Merkle chains and Proof-of-Authority (PoA) consensus (matching the architecture of Amazon Managed Blockchain and Hyperledger Fabric).

2.2 Block Structure & Cryptographic Mining

Each block is cryptographically linked to its predecessor via SHA-256 hashing. Modifying any historical block alters its hash and immediately invalidates the entire chain:

```
class Block {
  constructor(index, timestamp, transactions, previousHash = '', nonce = 0) {
```

```
    this.index = index;
    this.timestamp = timestamp;
    this.transactions = transactions; // Custody transfers, Smart Contract events
    this.previousHash = previousHash;
    this.nonce = nonce;
    this.hash = this.calculateHash();
  }

  calculateHash() {
    const dataString = this.index + this.previousHash + this.timestamp +
                       JSON.stringify(this.transactions) + this.nonce;
    return crypto.createHash('sha256').update(dataString).digest('hex');
  }
}
```

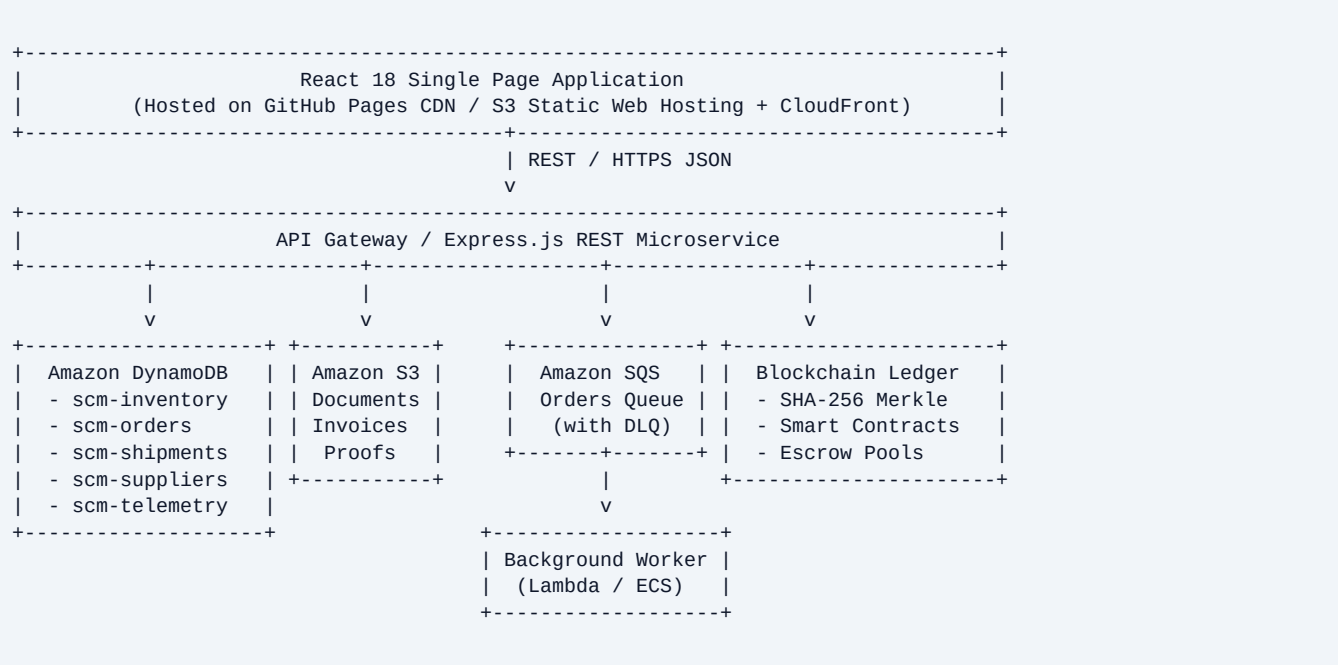
2.3 Automated Cold-Chain Escrow Smart Contracts

The system deploys the SC-COLD-CHAIN-SLA smart contract. As IoT sensors transmit temperature readings along shipping routes, an automated Oracle node evaluates the contract conditions. If the temperature exceeds safe limits (>8°C), the smart contract automatically freezes the escrow pool ($25,000 USD) and records an immutable breach transaction onto the blockchain.

3. System Architecture & AWS Cloud Topography

3.1 High-Level Architecture Topology

The architecture is designed around an **event-driven, decoupled microservices paradigm**. Client requests never directly execute slow disk transactions or synchronous vendor dependencies. Instead, requests enter scalable AWS buffers that absorb massive volume spikes with zero data loss.



3.2 Deep Dive: Individual AWS Services & Roles

- **Amazon DynamoDB (Operational Store):** Chosen for predictable single-digit millisecond latency at any throughput scale. Uses composite primary keys (Partition Key + Sort Key) and Global Secondary Indexes (GSIs). An automated Time-to-Live (TTL) attribute is assigned to IoT telemetry records to automatically prune aging time-series without incurred write/delete IOPS costs.
- **Amazon SQS (Decoupled Message Queue):** Acts as the shock-absorber for order placement. Ingests high-concurrency order spikes into a durable buffer. Configured with a visibility timeout of 60 seconds, long-polling (10s) to minimize empty receives, and a Dead-Letter Queue (DLQ) with maxReceiveCount = 3 to quarantine malformed messages without blocking the queue.
- **Amazon SNS (Pub/Sub Notification Engine):** Provides instant multi-channel alert fanout. Subscribed endpoints (such as cold-chain logistics operations emails, mobile SMS, or automated webhooks) receive immediate notification when IoT sensors detect temperature violations or stock is below safety levels.
- **Amazon S3 (Immutable Document Lake):** Provides 99.999999999% (11 9s) durability for compliance artifacts. Stores digital purchase invoices, bills of lading, and signature receipts. Server-Side Encryption (SSE-S3 AES-256) and bucket versioning protect documents against tampering or accidental deletion.
- **Amazon Managed Blockchain (AMB):** Provides immutable multi-party ledger storage, validating digital HMAC-SHA-256 custody signatures across suppliers, logistics carriers, and warehouse inspectors.

4. Step-by-Step Engineering Journey (From Zero to Production)

The project was engineered modularly across distinct phases, with atomic Git commits marking each milestone. Below is the exact chronological implementation journey executed by the development team:

Phase 1: Project Scaffolding, Governance & Root CI/CD

Git commit: eb4ecc5
Initialized git repository, root package.json workspace scripts, exhaustive .gitignore (preventing credential leakage), MIT License, and GitHub Actions CI/CD workflow (.github/workflows/ci-cd.yml) validating Node syntax, npm dependencies, and Terraform formatting.

Phase 2: Infrastructure as Code (HashiCorp Terraform)

Git commit: c6e5c57
Authored terraform/main.tf, variables.tf, lambda.tf, and outputs.tf. Created declarative specifications for 5 DynamoDB tables, S3 document storage with public access blocks and SSE-AES256, SQS queue with redrive policy targeting a dedicated Dead-Letter Queue (DLQ), and SNS alerts topic. Added least-privilege IAM policies granting Lambda strictly necessary DynamoDB, SQS, S3, and SNS permissions.

Phase 3: Backend REST API with AWS SDK v3 & Hybrid Mock Adapter

Git commit: f46c2c1
Built express REST API in server/src/. Implemented AWS SDK v3 clients (@aws-sdk/client-dynamodb, lib-dynamodb, client-sqs, client-sns, client-s3). Engineered an intelligent hybrid layer (aws.js): when AWS credentials exist, it communicates with live cloud resources; when offline or in demo mode, it activates a zero-cost local in-memory AWS Adapter that accurately replicates DynamoDB query semantics, SQS worker polling, and SNS alerts.

Phase 4: Frontend Supply Chain Control Tower (React 18 + Vite)

Git commit: 134c0ef
Constructed responsive UI with Tailwind CSS, Lucide icons, and Recharts. Implemented SCM Control Tower, Multi-facility Warehouse inventory viewer, Asynchronous SQS order queue visualizer with interactive worker polling, live IoT Cold-Chain sensor telemetry monitor with temperature charts, and an interactive AWS Architecture documentation explorer.

Phase 5: Containerization, CLI Simulator & Seed Data

Git commit: 294ada6
Authored multi-stage Dockerfiles for both server (Node 20 Alpine) and client (Nginx reverse-proxy on Alpine). Added docker-compose.yml for one-command orchestration. Developed scripts/simulate-iot.js (streaming telemetry along a simulated highway route) and scripts/seed-data.js.

Phase 6: GitHub Pages Automated Deployment & In-Browser Cloud Simulator

Git commits: 63bf77e, 4dba302, fdfe430
Engineered a dedicated GitHub Actions workflow (.github/workflows/deploy-gh-pages.yml) using peaceiris/actions-gh-pages@v4. Added an in-browser client-side simulator in client/src/services/api.js so the live web app functions 100% interactively on GitHub Pages without requiring an external backend!

Phase 7: Cryptographic Blockchain Ledger & Smart Contracts

Added blockchainService.js, blockchainController.js, and client/src/pages/Blockchain.jsx with live block explorer, Merkle SHA-256 chain verification, automated escrow execution, and digital custody handover signatures.

5. Cloud Security, Secrets Management & IaC Governance

5.1 Why Terraform Code and CLI Commands Are Safe in Public Repositories

A common concern among developers is whether publishing Terraform code and CLI commands (e.g. terraform apply) poses a security vulnerability. **The answer is an unequivocal NO.**

- **Infrastructure Code is a Blueprint, Not a Key:** HCL code describes WHAT resources to create (e.g., 'create an S3 bucket named xyz'). It does not grant anyone access to your AWS account. Only an authenticated AWS Identity (configured locally on the developer's computer) has access.
- **Strict .gitignore Isolation:** All files containing actual runtime credentials or state are permanently ignored by Git: .env, *.tfstate, *.tfstate.backup, and *.tfvars. No AWS access keys or secret keys ever touch Git.
- **Least-Privilege IAM Roles:** Terraform creates IAM execution policies with explicit Resource ARNs. The Lambda service role can only access the specific SCM tables and queues, eliminating lateral privilege escalation.
- **S3 Public Access Block:** The S3 bucket configuration explicitly enables all four Amazon S3 Public Access Blocks (BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, RestrictPublicBuckets) alongside SSE-S3 AES-256 encryption.

6. Developer Runbook & Local Execution Guide

6.1 Running the Project Locally

The project includes an intelligent mock AWS adapter so that any engineer can run the system immediately without incurring AWS billing or configuring credentials:

```
# 1. Start Backend API Server (Port 5000)
cd server && npm install && npm run dev

# 2. Start Frontend Web Dashboard (Port 5173) in a second terminal
cd client && npm install && npm run dev

# 3. (Optional) Run Live IoT Vehicle Telemetry Stream in a third terminal
node scripts/simulate-iot.js
```

6.2 Viewing the Documentation

- **Developer_Manual.html**: Double-click to open immediately in Google Chrome, Microsoft Edge, or Safari.
- **Multipurpose_SCM_Complete_Developer_Manual.pdf**: Open with Adobe Acrobat Reader, Chrome, or Edge (Right Click ➔ Open with ➔ Chrome).
- **Multipurpose_SCM_AWS_Architecture.pptx**: Open directly in Microsoft PowerPoint or Google Slides.